

13-后端Web进阶(AOP)



如果大家在自学的过程中，时间紧迫、没有方向、经常遇到难以解决的Bug、没有亮眼的项目、缺少AI大模型经验，而又想快速系统化的学习，高薪就业的小伙伴们，大家都可以[直接点我](#)，了解一下我们系统化的课程体系。

接下来我们进入到AOP的学习。AOP也是spring框架的第二大核心，我们先来学习AOP的基础。

什么是AOP?

AOP: Aspect Oriented Programming（面向切面编程、面向方面编程），其实说白了，面向切面编程就是面向特定方法编程。

那什么又是面向方法编程呢，为什么又需要面向方法编程呢？

来，我们举个例子做一个说明：

比如，我们这里有一个项目，项目中开发了很多的业务功能。然而有一些业务功能执行效率比较低，执行耗时较长，我们需要针对于这些业务方法进行优化。那首先第一步就需要定位出执行耗时比较长的业务方法，再针对于业务方法再来进行优化。

```
DeptServiceImpl

public List<Dept> list(){
    List<Dept> deptList = deptMapper.list();
    return deptList;
}

public void delete(Integer id) {
    deptMapper.delete(id);
}

public Dept getById(Integer id) {
    Dept dept = deptMapper.getById(id);
    return dept;
}
```

此时我们就需要统计当前这个项目当中每一个业务方法的执行耗时。那么统计每一个业务方法的执行耗时该怎么实现？

可能多数人首先想到的就是在每一个业务方法运行之前，记录这个方法运行的开始时间。在这个方法运行完毕之后，再来记录这个方法运行的结束时间。拿结束时间减去开始时间，不就是这个方法的执行耗时吗。



```
原始方式

public List<Dept> list(){
    long beginTime = System.currentTimeMillis();
    List<Dept> deptList = deptMapper.list();
    long endTime = System.currentTimeMillis();
    log.info(" 执行耗时: {} ms", endTime - beginTime);
    return deptList;
}

public void delete(Integer id) {
    long beginTime = System.currentTimeMillis();
    deptMapper.delete(id);
    long endTime = System.currentTimeMillis();
    log.info(" 执行耗时: {} ms", endTime - beginTime);
}

public Dept getById(Integer id){
    long beginTime = System.currentTimeMillis();
    Dept dept = deptMapper.getById(id);
    long endTime = System.currentTimeMillis();
    log.info(" 执行耗时: {} ms", endTime - beginTime);
    return dept;
}

//其他方法省略...
```

而这个功能如果通过AOP来实现，我们只需要单独定义下面这一小段代码即可，不需要修改原始的任何业务方法即可记录每一个业务方法的执行耗时。

AOP方式

```
@Slf4j
@Aspect
@Component
public class RecordTimeAspect {
    @Around("execution(* com.itheima.service.*(..))")
    public Object recordTime(ProceedingJoinPoint pjp) {
        long beginTime = System.currentTimeMillis();
        Object result = pjp.proceed();
        long endTime = System.currentTimeMillis();
        log.info("执行耗时: {}ms", endTime - beginTime);
        return result;
    }
}
```

所以，AOP的优势主要体现在以下四个方面：

- 减少重复代码：不需要在业务方法中定义大量的重复性的代码，只需要将重复性的代码抽取到AOP程序中即可。
- 代码无侵入：在基于AOP实现这些业务功能时，对原有的业务代码是没有任何侵入的，不需要修改任何的代码。
- 提高开发效率
- 维护方便

✓ AOP是一种思想，而在Spring框架中，对这种思想进行了实现，那我们要学习的就是Spring AOP。

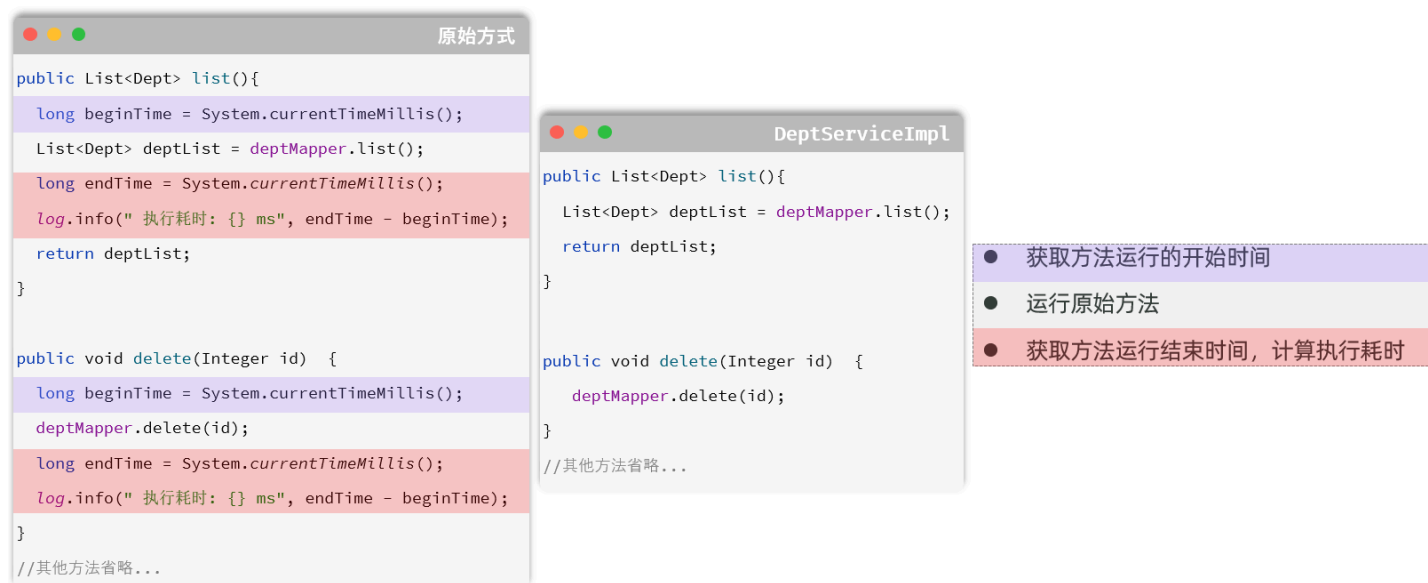
1. AOP基础

1.1 AOP入门

在了解了什么是AOP后，我们下面通过一个快速入门程序，体验下AOP的开发，并掌握Spring中AOP的开发步骤。

- 需求：统计部门管理各个业务层方法执行耗时。
- 原始方式：

在原始的实现方式中，我们需要在业务层的也一个方法执行执行，获取方法运行的开始时间；然后运行原始的方法逻辑；最后在每一个方法运行结束时，获取方法运行结束时间，计算执行耗时。



• SpringAOP实现步骤:

为演示方便，可以直接导入资料中提供的 `springboot-aop-quickstart` 项目工程

1). 导入依赖：在 pom.xml 文件中导入 AOP 的依赖

代码块

```
1 <dependency>
2     <groupId>org.springframework.boot</groupId>
3     <artifactId>spring-boot-starter-aop</artifactId>
4 </dependency>
```

2). 编写AOP程序：针对于特定方法根据业务需要进行编程

代码块

```
1 @Component
2 @Aspect //当前类为切面类
3 @Slf4j
4 public class RecordTimeAspect {
5
6     @Around("execution(* com.itheima.service.impl.DeptServiceImpl.*(..))")
7     public Object recordTime(ProceedingJoinPoint pjp) throws Throwable {
8         //记录方法执行开始时间
9         long begin = System.currentTimeMillis();
10
```

```
11 //执行原始方法
12 Object result = pjp.proceed();
13
14 //记录方法执行结束时间
15 long end = System.currentTimeMillis();
16
17 //计算方法执行耗时
18 log.info("方法执行耗时: {}毫秒",end-begin);
19 return result;
20 }
21 }
```

重新启动SpringBoot服务，打开浏览器访问部门管理的功能进行测试：

Tlias智能学习辅助系统

修改密码 | 退出登录【未读】

首页

班级学员管理

系统信息管理

部门管理

员工管理

数据统计管理

部门管理

新增

序号	部门名称	最后修改时间	操作
1	学工部	2024-07-25T09:47:40	修改 删除
2	教研部	2024-08-09T15:17:04	修改 删除
3	咨询部	2024-07-30T21:26:24	修改 删除
4	人事部	2024-08-14T16:17:43	修改 删除
5	近派使算他	2024-08-15T09:36:31	修改 删除

我们可以看到，在控制台中输出了方法的执行耗时：

Services

Spring Boot

SpringbootAopQuickstart

Console

Actuator

2024-07-25 09:47:15.481+08:00 INFO 9836 --- [nio-8080-exec-1] com.itheima.aop.RecordTimeAspect : 方法执行耗时: 375毫秒

我们通过AOP入门程序完成了业务方法执行耗时的统计，那其实AOP的功能远不止于此，常见的应用场景如下：

- 记录系统的操作日志
- 权限控制

- 事务管理：我们前面所讲解的Spring事务管理，底层其实也是通过AOP来实现的，只要添加@Transactional注解之后，AOP程序自动会在原始方法运行前先来开启事务，在原始方法运行完毕之后提交或回滚事务

这些都是AOP应用的典型场景。

通过入门程序，我们也应该感受到了AOP面向切面编程的一些优势：

- 代码无侵入：没有修改原始的业务方法，就已经对原始的业务方法进行了功能的增强或者是功能的改变
- 减少了重复代码
- 提高开发效率
- 维护方便

1.2 AOP核心概念

通过SpringAOP的快速入门，感受了一下AOP面向切面编程的开发方式。下面我们再来学习AOP当中涉及到的一些核心概念。

- **连接点：JoinPoint**，可以被AOP控制的方法（暗含方法执行时的相关信息）
 - 连接点指的是可以被aop控制的方法。例如：入门程序当中所有的业务方法都是可以被aop控制的方法。
 - 在SpringAOP提供的JoinPoint当中，封装了连接点方法在执行时的相关信息。（后面会有具体的讲解）

```

@Service
public class DeptServiceImpl implements DeptService {
    @Autowired
    private DeptMapper deptMapper;

    @Override
    public List<Dept> list() {
        List<Dept> deptList = deptMapper.list();
        return deptList;
    }

    @Override
    public void delete(Integer id) {
        deptMapper.delete(id);
    }

    @Override
    public void save(Dept dept) {
        dept.setCreateTime(LocalDateTime.now());
        dept.setUpdateTime(LocalDateTime.now());
        deptMapper.save(dept);
    }

    @Override
    public Dept getById(Integer id) {
        return deptMapper.getById(id);
    }
}

```

连接点

- **通知：Advice**，指哪些重复的逻辑，也就是共性功能（最终体现为一个方法）
 - 在入门程序中是需要统计各个业务方法的执行耗时的，此时我们就需要在这些业务方法运行开始之前，先记录这个方法运行的开始时间，在每一个业务方法运行结束的时候，再来记录这个方法运行的结束时间。
 - 是在AOP面向切面编程当中，我们只需要将这部分重复的代码逻辑抽取出来单独定义。抽取出来的这一部分重复的逻辑，也就是共性的功能。

```

@Slf4j
@Aspect
@Component
public class RecordTimeAspect {

    @Around("execution(* com.itheima.service.impl.DeptServiceImpl.*(..))")
    public Object recordTime(ProceedingJoinPoint pjp) throws Throwable {
        long beginTime = System.currentTimeMillis();
        Object result = pjp.proceed();
        long endTime = System.currentTimeMillis();
        log.info(" 执行耗时: {} ms", endTime - beginTime);
        return result;
    }
}

```

通知

- **切入点：PointCut**，匹配连接点的条件，通知仅会在切入点方法执行时被应用。
 - 在通知当中，我们所定义的共性功能到底要应用在哪些方法上？此时就涉及到了切入点pointcut概念。切入点指的是匹配连接点的条件。通知仅会在切入点方法运行时才会被应用。
 - 在aop的开发当中，我们通常会通过一个切入点表达式来描述切入点(后面会有详解)。

```

@Slf4j
@Aspect
@Component
public class RecordTimeAspect {  切入点表达式

    @Around("execution(* com.itheima.service.impl.DeptServiceImpl.*(..))")
    public Object recordTime(ProceedingJoinPoint pjp) throws Throwable {
        long beginTime = System.currentTimeMillis();
        Object result = pjp.proceed();
        long endTime = System.currentTimeMillis();
        log.info(" 执行耗时: {} ms", endTime - beginTime);
        return result;
    }
}

```

通知

- 假如：切入点表达式改为DeptServiceImpl.list()，此时就代表仅仅只有list这一个方法是切入点。只有list()方法在运行的时候才会应用通知。
- **切面：Aspect**，描述通知与切入点的对应关系（通知+切入点）

当通知和切入点结合在一起，就形成了一个切面。通过切面就能够描述当前aop程序需要针对于哪个原始方法，在什么时候执行什么样的操作。


```
@Slf4j
@Aspect
@Component
public class RecordTimeAspect {    切入点表达式

    @Around("execution(* com.itheima.service.impl.DeptServiceImpl.*(..))")
    public Object recordTime(ProceedingJoinPoint pjp) throws Throwable {
        long beginTime = System.currentTimeMillis();
        Object result = pjp.proceed();
        long endTime = System.currentTimeMillis();
        log.info(" 执行耗时: {} ms", endTime - beginTime);
        return result;
    }
}
```

通知 切面

而切面所在的类，称之为切面类（被 `@Aspect` 注解标识的类）。

- **目标对象：Target**，通知所应用的对象

目标对象指的就是通知所应用的对象，我们就称之为目标对象。

@Service

目标对象

public class DeptServiceImpl implements DeptService {

@Autowired

private DeptMapper deptMapper;

@Override

public List<Dept> list() {

切入点

list<Dept> deptList = deptMapper.list();

return deptList;

}

@Override

public void delete(Integer id) {

deptMapper.delete(id);

}

@Override

连接点

public void save(Dept dept) {

dept.setCreateTime(LocalDateTime.now());

dept.setUpdateTime(LocalDateTime.now());

deptMapper.save(dept);

}

@Override

public Dept getById(Integer id) {

return deptMapper.getById(id);

}

}

AOP的核心概念我们介绍完毕之后，接下来我们再来分析一下我们所定义的通知是如何与目标对象结合在一起，对目标对象当中的方法进行功能增强的。

```
@Aspect
public class RecordTimeAspect {
    @Around("execution(* com.itheima.service.impl.*(..))")
    public Object recordTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long begin = System.currentTimeMillis();
        Object result = joinPoint.proceed(); //调用原始操作
        long end = System.currentTimeMillis();
        log.info("执行耗时: {} ms", (end-begin));
        return result;
    }
}
```

```
public class DeptServiceImpl implements DeptService {
    @Autowired
    private DeptMapper deptMapper;
    @Override
    public List<Dept> list() {
        List<Dept> deptList = deptMapper.list();
        return deptList;
    }
}
```

目标对象

```
public class DeptController {
    @Autowired
    private DeptService deptService;
    @GetMapping
    public Result list(){
        List<Dept> deptList = deptService.list();
        return Result.success(deptList);
    }
}
```

```
public class DeptServiceProxy implements DeptService {
    @Override
    public List<Dept> list() {
        long begin = System.currentTimeMillis();
        List<Dept> deptList = 目标对象.list(); //伪代码
        long end = System.currentTimeMillis();
        log.info("执行耗时: {} ms", (end-begin));
        return deptList;
    }
}
```

代理对象

Spring的AOP底层是基于动态代理技术来实现的，也就是说在程序运行的时候，会自动的基于动态代理技术为目标对象生成一个对应的代理对象。在代理对象当中就会对目标对象当中的原始方法进行功能的增强。

✓ SpringAOP 旨在管理bean对象的过程中，主要通过底层的动态代理机制，对特定的方法进行编程。

2. AOP进阶

AOP的基础知识学习完之后，下面我们对AOP当中的各个细节进行详细的学习。主要分为4个部分：

1. 通知类型
2. 通知顺序
3. 切入点表达式

我们先来学习第一部分通知类型。

2.1 通知类型

在入门程序当中，我们已经使用了一种功能最为强大的通知类型：Around环绕通知。

代码块

```
1 @Component
2 @Aspect //当前类为切面类
```

```

3  @Slf4j
4  public class TimeAspect {
5
6      @Around("execution(* com.itheima.service.impl.DeptServiceImpl.*(..))")
7      public Object recordTime(ProceedingJoinPoint pjp) throws Throwable {
8          //记录方法执行开始时间
9          long begin = System.currentTimeMillis();
10         //执行原始方法
11         Object result = pjp.proceed();
12         //记录方法执行结束时间
13         long end = System.currentTimeMillis();
14         //计算方法执行耗时
15         log.info("方法执行耗时: {}毫秒", end - begin);
16         return result;
17     }
18 }

```

只要我们在通知方法上加上了 `@Around` 注解，就代表当前通知是一个环绕通知。

Spring AOP 通知类型	
@Around	环绕通知，此注解标注的通知方法在目标方法前、后都被执行
@Before	前置通知，此注解标注的通知方法在目标方法前被执行
@After	后置通知，此注解标注的通知方法在目标方法后被执行，无论是否有异常都会执行
@AfterReturning	返回后通知，此注解标注的通知方法在目标方法后被执行，有异常不会执行
@AfterThrowing	异常后通知，此注解标注的通知方法发生异常后执行

下面我们通过代码演示，来加深对于不同通知类型的理解：

代码块

```

1  @Slf4j
2  @Component
3  @Aspect
4  public class MyAspect1 {
5      //前置通知
6      @Before("execution(* com.itheima.service.*.*(..))")
7      public void before(JoinPoint joinPoint){
8          log.info("before ...");
9      }

```

```

10     }
11
12     //环绕通知
13     @Around("execution(* com.itheima.service.*.*(..))")
14     public Object around(ProceedingJoinPoint proceedingJoinPoint) throws
    Throwable {
15         log.info("around before ...");
16
17         //调用目标对象的原始方法执行
18         Object result = proceedingJoinPoint.proceed();
19
20         //原始方法如果执行时有异常，环绕通知中的后置代码不会在执行了
21
22         log.info("around after ...");
23         return result;
24     }
25
26     //后置通知
27     @After("execution(* com.itheima.service.*.*(..))")
28     public void after(JoinPoint joinPoint){
29         log.info("after ...");
30     }
31
32     //返回后通知（程序在正常执行的情况下，会执行的后置通知）
33     @AfterReturning("execution(* com.itheima.service.*.*(..))")
34     public void afterReturning(JoinPoint joinPoint){
35         log.info("afterReturning ...");
36     }
37
38     //异常通知（程序在出现异常的情况下，执行的后置通知）
39     @AfterThrowing("execution(* com.itheima.service.*.*(..))")
40     public void afterThrowing(JoinPoint joinPoint){
41         log.info("afterThrowing ...");
42     }
43 }
44

```

重新启动SpringBoot服务，进行测试：

1). 没有异常情况下：

使用 Apifox 测试查询所有部门数据



查看idea中控制台日志输出：

```
12-09T10:52:50.998+08:00 INFO 1492 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
12-09T10:52:51.022+08:00 INFO 1492 --- [nio-8080-exec-1] com.itheima.aop.MyAspect1 : around before ...
12-09T10:52:51.023+08:00 INFO 1492 --- [nio-8080-exec-1] com.itheima.aop.MyAspect1 : before ...
12-09T10:52:51.037+08:00 INFO 1492 --- [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
12-09T10:52:51.366+08:00 INFO 1492 --- [nio-8080-exec-1] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.Conn
12-09T10:52:51.368+08:00 INFO 1492 --- [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
12-09T10:52:51.404+08:00 INFO 1492 --- [nio-8080-exec-1] com.itheima.aop.MyAspect1 : afterReturning ...
12-09T10:52:51.405+08:00 INFO 1492 --- [nio-8080-exec-1] com.itheima.aop.MyAspect1 : after ...
12-09T10:52:51.405+08:00 INFO 1492 --- [nio-8080-exec-1] com.itheima.aop.MyAspect1 : around after ...
```

程序没有发生异常的情况下，@AfterThrowing标识的通知方法不会执行。

2). 出现异常情况下：

修改DeptServiceImpl业务实现类中的代码：添加异常

代码块

```
1 @Slf4j
2 @Service
3 public class DeptServiceImpl implements DeptService {
4     @Autowired
5     private DeptMapper deptMapper;
6
7     @Override
8     public List<Dept> list() {
9
10         List<Dept> deptList = deptMapper.list();
11         //模拟异常
12         int num = 10/0;
```

```
13         return deptList;
14     }
15
16     //省略其他代码...
17 }
```

重新启动SpringBoot服务，测试发生异常情况下通知的执行：



GET http://localhost:8080/depts

Params Body Headers Cookies Auth 前置操作 后置操作 设置

Query 参数

参数名	参数值	类型	说明
添加参数			

Body Cookie Header 5 控制台 实际请求

Pretty Raw Preview Visualize JSON utf8

```
1 {
2   "code": 1,
3   "msg": "success",
4   "data": [
5     {
6       "id": 1,
7       "name": "学工部",
8       "createTime": "2023-09-25T09:47:40",
9       "updateTime": "2023-12-09T10:13:02"
10    },
11    {
12      "id": 2,
13      "name": "教研部",
14      "createTime": "2023-09-25T09:47:40",
15      "updateTime": "2023-10-09T15:17:04"
16    }
17  ]
18 }
```

校验响应 OK (200)

200 571 ms 624 B

返回数据结构与接口定义不一致

1. \$.data 应当是 object,null 类型

查看idea中控制台日志输出

```
12-09T10:54:30.611+08:00 INFO 11852 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
12-09T10:54:30.635+08:00 INFO 11852 --- [nio-8080-exec-2] com.itheima.aop.MyAspect1 : around before ...
12-09T10:54:30.635+08:00 INFO 11852 --- [nio-8080-exec-2] com.itheima.aop.MyAspect1 : before ...
12-09T10:54:30.649+08:00 INFO 11852 --- [nio-8080-exec-2] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
12-09T10:54:30.986+08:00 INFO 11852 --- [nio-8080-exec-2] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.Conr
12-09T10:54:30.987+08:00 INFO 11852 --- [nio-8080-exec-2] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
12-09T10:54:31.022+08:00 INFO 11852 --- [nio-8080-exec-2] com.itheima.aop.MyAspect1 : afterThrowing ...
12-09T10:54:31.022+08:00 INFO 11852 --- [nio-8080-exec-2] com.itheima.aop.MyAspect1 : after ...
12-09T10:54:31.027+08:00 ERROR 11852 --- [nio-8080-exec-2] o.a.c.c.C.[.][/].[dispatcherServlet] : Servlet.service() for servlet [dispatcherServlet] in c

java.lang.ArithmeticException: Create breakpoint : / by zero
    at com.itheima.service.impl.DeptServiceImpl.list(DeptServiceImpl.java:22) ~[classes/:na] <2 internal lines>
    at org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:343) ~[spring-aop-6.0.14.jar:6.0.14]
```

程序发生异常的情况下：

- @AfterReturning标识的通知方法不会执行，@AfterThrowing标识的通知方法执行了
- @Around环绕通知中原始方法调用时有异常，通知中的环绕后的代码逻辑也不会执行了（因为原始方法调用已经出异常了）

✓ 在使用通知时的注意事项：

- @Around环绕通知需要自己调用 `ProceedingJoinPoint.proceed()` 来让原始方法执行，其他通知不需要考虑目标方法执行
- @Around环绕通知方法的返回值，必须指定为Object，来接收原始方法的返回值，否则原始方法执行完毕，是获取不到返回值的。

五种常见的通知类型，我们已经测试完毕了，此时我们再来看一下刚才所编写的代码，有什么问题吗？

代码块

```
1 //前置通知
2 @Before("execution(* com.itheima.service.*.*(..))")
3
4 //环绕通知
5 @Around("execution(* com.itheima.service.*.*(..))")
6
7 //后置通知
8 @After("execution(* com.itheima.service.*.*(..))")
9
10 //返回后通知（程序在正常执行的情况下，会执行的后置通知）
11 @AfterReturning("execution(* com.itheima.service.*.*(..))")
12
13 //异常通知（程序在出现异常的情况下，执行的后置通知）
14 @AfterThrowing("execution(* com.itheima.service.*.*(..))")
```

我们发现啊，每一个注解里面都指定了切入点表达式，而且这些切入点表达式都一模一样。此时我们的代码当中就存在了大量的重复性的切入点表达式，假如此时切入点表达式需要变动，就需要将所有的切入点表达式一个一个的来改动，就变得非常繁琐了。

怎么来解决这个切入点表达式重复的问题？ 答案就是：**抽取**

Spring提供了 `@PointCut` 注解，该注解的作用是将公共的切入点表达式抽取出来，需要用到时引用该切入点表达式即可。

代码块

```
1 @Slf4j
2 @Component
3 @Aspect
4 public class MyAspect1 {
```



```
5
6    //切入点方法（公共的切入点表达式）
7    @Pointcut("execution(* com.itheima.service.*(..))")
8    private void pt(){}
9
10   //前置通知（引用切入点）
11   @Before("pt()")
12   public void before(JoinPoint joinPoint){
13       log.info("before ...");
14
15   }
16
17   //环绕通知
18   @Around("pt()")
19   public Object around(ProceedingJoinPoint proceedingJoinPoint) throws
Throwable {
20       log.info("around before ...");
21
22       //调用目标对象的原始方法执行
23       Object result = proceedingJoinPoint.proceed();
24       //原始方法在执行时：发生异常
25       //后续代码不在执行
26
27       log.info("around after ...");
28       return result;
29   }
30
31   //后置通知
32   @After("pt()")
33   public void after(JoinPoint joinPoint){
34       log.info("after ...");
35   }
36
37   //返回后通知（程序在正常执行的情况下，会执行的后置通知）
38   @AfterReturning("pt()")
39   public void afterReturning(JoinPoint joinPoint){
40       log.info("afterReturning ...");
41   }
42
43   //异常通知（程序在出现异常的情况下，执行的后置通知）
44   @AfterThrowing("pt()")
45   public void afterThrowing(JoinPoint joinPoint){
46       log.info("afterThrowing ...");
47   }
48 }
```

需要注意的是：当切入点方法使用 `private` 修饰时，仅能在当前切面类中引用该表达式，当外部其他切面类中也要引用当前类中的切入点表达式，就需要把 `private` 改为 `public`，而在引用的时候，具体的语法为：

代码块

```
1  @Slf4j
2  @Component
3  @Aspect
4  public class MyAspect2 {
5      //引用MyAspect1切面类中的切入点表达式
6      @Before("com.itheima.aspect.MyAspect1.pt()")
7      public void before(){
8          log.info("MyAspect2 -> before ...");
9      }
10 }
```

2.2 通知顺序

讲解完了Spring中AOP所支持的5种通知类型之后，接下来我们再来研究通知的执行顺序。

当在项目开发当中，我们定义了多个切面类，而多个切面类中多个切入点都匹配到了同一个目标方法。此时当目标方法在运行的时候，这多个切面类当中的这些通知方法都会运行。

此时我们就有一个疑问，这多个通知方法到底哪个先运行，哪个后运行？下面我们通过程序来验证（这里呢，我们就定义两种类型的通知进行测试，一种是前置通知 `@Before`，一种是后置通知 `@After`）

定义多个切面类：

代码块

```
1  @Slf4j
2  @Component
3  @Aspect
4  public class MyAspect2 {
5      //前置通知
6      @Before("execution(* com.itheima.service.*.*(..))")
7      public void before(){
8          log.info("MyAspect2 -> before ...");
9      }
10 }
```

```
10
11     //后置通知
12     @After("execution(* com.itheima.service.*.*(..))")
13     public void after(){
14         log.info("MyAspect2 -> after ...");
15     }
16 }
```

代码块

```
1  @Slf4j
2  @Component
3  @Aspect
4  public class MyAspect3 {
5      //前置通知
6      @Before("execution(* com.itheima.service.*.*(..))")
7      public void before(){
8          log.info("MyAspect3 -> before ...");
9      }
10
11     //后置通知
12     @After("execution(* com.itheima.service.*.*(..))")
13     public void after(){
14         log.info("MyAspect3 -> after ...");
15     }
16 }
```

代码块

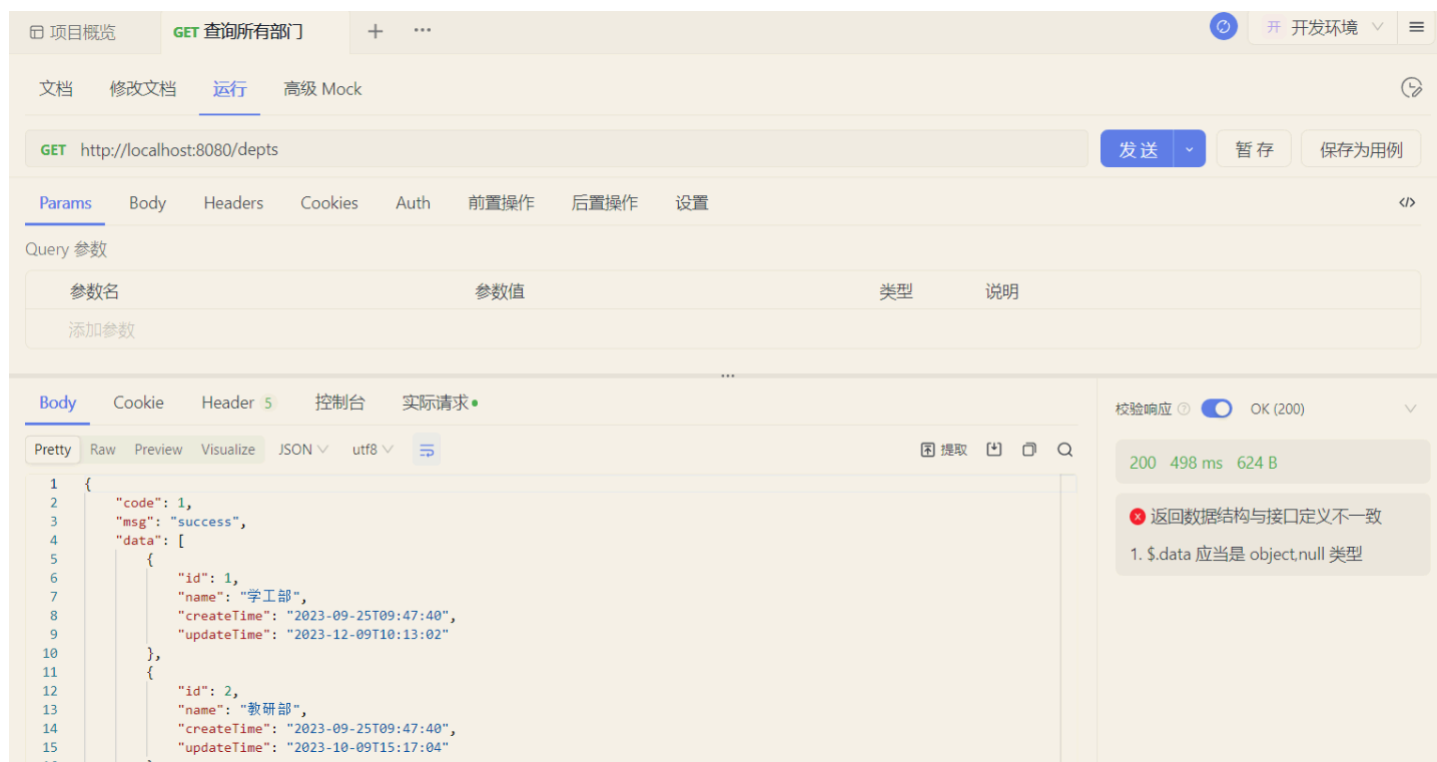
```
1  @Slf4j
2  @Component
3  @Aspect
4  public class MyAspect4 {
5      //前置通知
6      @Before("execution(* com.itheima.service.*.*(..))")
7      public void before(){
8          log.info("MyAspect4 -> before ...");
9      }
10
11     //后置通知
12     @After("execution(* com.itheima.service.*.*(..))")
13     public void after(){
14         log.info("MyAspect4 -> after ...");
15     }
16 }
```

重新启动SpringBoot服务，测试通知的执行顺序：

备注：

1. 把DeptServiceImpl实现类中模拟异常的代码删除或注释掉。
2. 注释掉其他切面类(把 `@Aspect` 注释即可)，仅保留MyAspect2、MyAspect3、MyAspect4，这样就可以清晰看到执行的结果，而不被其他切面类干扰。

使用 Apifox 测试查询所有部门数据。



查看idea中控制台日志输出



- 通过以上程序运行可以看出在不同切面类中，默认按照切面类的类名字母排序：

- 目标方法前的通知方法：字母排名靠前的先执行
- 目标方法后的通知方法：字母排名靠前的后执行

如果我们想控制通知的执行顺序有两种方式：

1. 修改切面类的类名（这种方式非常繁琐、而且不便管理）
2. 使用Spring提供的 `@Order` 注解

使用@Order注解，控制通知的执行顺序：

代码块

```
1  @Slf4j
2  @Component
3  @Aspect
4  @Order(2) //切面类的执行顺序（前置通知：数字越小先执行；后置通知：数字越小越后执行）
5  public class MyAspect2 {
6      //前置通知
7      @Before("execution(* com.itheima.service.*.*(..))")
8      public void before(){
9          log.info("MyAspect2 -> before ...");
10     }
11
12     //后置通知
13     @After("execution(* com.itheima.service.*.*(..))")
14     public void after(){
15         log.info("MyAspect2 -> after ...");
16     }
17 }
```

代码块

```
1  @Slf4j
2  @Component
3  @Aspect
4  @Order(3) //切面类的执行顺序（前置通知：数字越小先执行；后置通知：数字越小越后执行）
5  public class MyAspect3 {
6      //前置通知
7      @Before("execution(* com.itheima.service.*.*(..))")
8      public void before(){
9          log.info("MyAspect3 -> before ...");
10     }
11
12     //后置通知
13     @After("execution(* com.itheima.service.*.*(..))")
14     public void after(){
15         log.info("MyAspect3 -> after ...");
16     }
17 }
```

```
16     }
17 }
```

代码块

```
1  @Slf4j
2  @Component
3  @Aspect
4  @Order(1) //切面类的执行顺序（前置通知：数字越小先执行；后置通知：数字越小越后执行）
5  public class MyAspect4 {
6      //前置通知
7      @Before("execution(* com.itheima.service.*.*(..))")
8      public void before(){
9          log.info("MyAspect4 -> before ...");
10     }
11
12     //后置通知
13     @After("execution(* com.itheima.service.*.*(..))")
14     public void after(){
15         log.info("MyAspect4 -> after ...");
16     }
17 }
```

重新启动SpringBoot服务，测试通知执行顺序：

```
12-09T11:00:52.930+08:00 INFO 1384 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
12-09T11:00:52.931+08:00 INFO 1384 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
12-09T11:00:52.958+08:00 INFO 1384 --- [nio-8080-exec-2] com.itheima.aop.MyAspect4 : MyAspect4 -> before ...
12-09T11:00:52.958+08:00 INFO 1384 --- [nio-8080-exec-2] com.itheima.aop.MyAspect2 : MyAspect2 -> before ...
12-09T11:00:52.958+08:00 INFO 1384 --- [nio-8080-exec-2] com.itheima.aop.MyAspect3 : MyAspect3 -> before ...
12-09T11:00:52.974+08:00 INFO 1384 --- [nio-8080-exec-2] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
12-09T11:00:53.290+08:00 INFO 1384 --- [nio-8080-exec-2] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.Conne
12-09T11:00:53.292+08:00 INFO 1384 --- [nio-8080-exec-2] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
12-09T11:00:53.329+08:00 INFO 1384 --- [nio-8080-exec-2] com.itheima.aop.MyAspect3 : MyAspect3 -> after ...
12-09T11:00:53.330+08:00 INFO 1384 --- [nio-8080-exec-2] com.itheima.aop.MyAspect2 : MyAspect2 -> after ...
12-09T11:00:53.330+08:00 INFO 1384 --- [nio-8080-exec-2] com.itheima.aop.MyAspect4 : MyAspect4 -> after ...
```



通知的执行顺序大家主要知道两点即可：

1. 不同的切面类当中，默认情况下通知的执行顺序是与切面类的类名字母排序是有关系的
2. 可以在切面类上面加上@Order注解，来控制不同的切面类通知的执行顺序

2.3 切入点表达式

从AOP的入门程序到现在，我们一直都在使用切入点表达式来描述切入点。下面我们就来详细的介绍一下切入点表达式的具体写法。

切入点表达式：描述切入点方法的一种表达式

- 作用：主要用来决定项目中的哪些方法需要加入通知
- 常见形式：
 - `execution(……)`：根据方法的签名来匹配

```
@Before("execution(public void com.itheima.service.impl.DeptServiceImpl.delete(java.lang.Integer))")
public void before(JoinPoint joinPoint){
```

- `@annotation(……)`：根据注解匹配

```
@Before("@annotation(com.itheima.anno.Log)")
public void before(){
```

首先我们先学习第一种最为常见的`execution`切入点表达式。

2.3.1 execution

`execution`主要根据方法的返回值、包名、类名、方法名、方法参数等信息来匹配，语法为：

代码块

```
1 execution(访问修饰符? 返回值 包名.类名.?方法名(方法参数) throws 异常?)
```

其中带 `?` 的表示可以省略的部分

- 访问修饰符：可省略（比如：`public`、`protected`）
- 包名.类名：可省略
- `throws 异常`：可省略（注意是方法上声明抛出的异常，不是实际抛出的异常）

示例：

代码块

```
1 @Before("execution(void
    com.itheima.service.impl.DeptServiceImpl.delete(java.lang.Integer))")
```

可以使用通配符描述切入点

- `*`：单个独立的任意符号，可以通配任意返回值、包名、类名、方法名、任意类型的一个参数，也可以通配包、类、方法名的一部分
- `..`：多个连续的任意符号，可以通配任意层级的包，或任意类型、任意个数的参数

切入点表达式的语法规则：

1. 方法的访问修饰符可以省略
2. 返回值可以使用 `*` 号代替（任意返回值类型）
3. 包名可以使用 `*` 号代替，代表任意包（一层包使用一个 `*`）
4. 使用 `..` 配置包名，标识此包以及此包下的所有子包
5. 类名可以使用 `*` 号代替，标识任意类
6. 方法名可以使用 `*` 号代替，表示任意方法
7. 可以使用 `*` 配置参数，一个任意类型的参数
8. 可以使用 `..` 配置参数，任意个任意类型的参数

切入点表达式示例

- 省略方法的修饰符号

代码块

```
1 execution(void  
    com.itheima.service.impl.DeptServiceImpl.delete(java.lang.Integer))
```

- 使用 `*` 代替返回值类型

代码块

```
1 execution(* com.itheima.service.impl.DeptServiceImpl.delete(java.lang.Integer))
```

- 使用 `*` 代替包名（一层包使用一个 `*`）

代码块

```
1 execution(* com.itheima.*.*.DeptServiceImpl.delete(java.lang.Integer))
```

- 使用 `..` 省略包名

代码块

```
1 execution(* com..DeptServiceImpl.delete(java.lang.Integer))
```

- 使用 `*` 代替类名

代码块

```
1 execution(* com..*.delete(java.lang.Integer))
```

- 使用 `*` 代替方法名

代码块

```
1 execution(* com..*.*(java.lang.Integer))
```

- 使用 `*` 代替参数

代码块

```
1 execution(* com.itheima.service.impl.DeptServiceImpl.delete(*))
```

- 使用 `..` 省略参数

代码块

```
1 execution(* com..*.*(..))
```

注意事项：

- 根据业务需要，可以使用 且（&&）、或（||）、非（!）来组合比较复杂的切入点表达式。

代码块

```
1 execution(* com.itheima.service.DeptService.list(..)) || execution(*  
com.itheima.service.DeptService.delete(..))
```

切入点表达式的书写建议：

- 所有业务方法名在命名时尽量规范，方便切入点表达式快速匹配。如：查询类方法都是 find 开头，更新类方法都是 update 开头

代码块

```
1 //业务类  
2 @Service  
3 public class DeptServiceImpl implements DeptService {  
4
```

```

5      public List<Dept> findAllDept() {
6          //省略代码...
7      }
8
9      public Dept findDeptById(Integer id) {
10         //省略代码...
11     }
12
13     public void updateDeptById(Integer id) {
14         //省略代码...
15     }
16
17     public void updateDeptByMoreCondition(Dept dept) {
18         //省略代码...
19     }
20     //其他代码...
21 }

```

- //匹配DeptServiceImpl类中以find开头的方法

代码块

```
1  execution(* com.itheima.service.impl.DeptServiceImpl.find*(..))
```

- 描述切入点方法通常基于接口描述，而不是直接描述实现类，增强拓展性

代码块

```
1  execution(* com.itheima.service.DeptService.*(..))
```

- 在满足业务需要的前提下，尽量缩小切入点的匹配范围。如：包名匹配尽量不使用 ..，使用 * 匹配单个包

代码块

```
1  execution(* com.itheima.*.*.DeptServiceImpl.find*(..))
```

切入点表达式书写建议：

- 所有业务方法名在命名时尽量规范，方便切入点表达式快速匹配。如：findXxx，updateXxx。
- 描述切入点方法通常基于接口描述，而不是直接描述实现类，增强拓展性。

- 在满足业务需要的前提下，尽量缩小切入点的匹配范围。如：包名尽量不使用..，使用 `*` 匹配单个包。

2.3.2 @annotation

已经学习了execution切入点表达式的语法。那么如果我们要匹配多个无规则的方法，比如：list()和delete()这两个方法。这个时候我们基于execution这种切入点表达式来描述就不是很方便了。而在之前我们是将两个切入点表达式组合在了一起完成的需求，这个是比较繁琐的。

我们可以借助于另一种切入点表达式 `@annotation` 来描述这一类的切入点，从而来简化切入点表达式的书写。

实现步骤：

1. 编写自定义注解
2. 在业务类要做为连接点的方法上添加自定义注解

自定义注解： `LogOperation`

代码块

```
1 @Target(ElementType.METHOD)
2 @Retention(RetentionPolicy.RUNTIME)
3 public @interface LogOperation{
4 }
```

业务类： `DeptServiceImpl`

代码块

```
1 @Slf4j
2 @Service
3 public class DeptServiceImpl implements DeptService {
4     @Autowired
5     private DeptMapper deptMapper;
6
7     @Override
8     @LogOperation //自定义注解（表示：当前方法属于目标方法）
9     public List<Dept> list() {
10         List<Dept> deptList = deptMapper.list();
```

```

11         //模拟异常
12         //int num = 10/0;
13         return deptList;
14     }
15
16     @Override
17     @LogOperation //自定义注解 (表示: 当前方法属于目标方法)
18     public void delete(Integer id) {
19         //1. 删除部门
20         deptMapper.delete(id);
21     }
22
23
24     @Override
25     public void save(Department dept) {
26         dept.setCreateTime(LocalDate.now());
27         dept.setUpdateTime(LocalDate.now());
28         deptMapper.save(dept);
29     }
30
31     @Override
32     public Department getById(Integer id) {
33         return deptMapper.getById(id);
34     }
35
36     @Override
37     public void update(Department dept) {
38         dept.setUpdateTime(LocalDate.now());
39         deptMapper.update(dept);
40     }
41 }

```

切面类

代码块

```

1  @Slf4j
2  @Component
3  @Aspect
4  public class MyAspect6 {
5      //针对list方法、delete方法进行前置通知和后置通知
6
7      //前置通知
8      @Before("@annotation(com.itheima.anno.LogOperation)")
9      public void before(){

```

```

10         log.info("MyAspect6 -> before ...");
11     }
12
13     //后置通知
14     @After("@annotation(com.itheima.anno.LogOperation)")
15     public void after(){
16         log.info("MyAspect6 -> after ...");
17     }
18 }

```

重启SpringBoot服务，测试查询所有部门数据，查看控制台日志：

```

2:57.307 INFO 15060 --- [nio-8080-exec-1] com.itheima.aspect.MyAspect6 : MyAspect6 -> before ...
SqlSession
.org.apache.ibatis.session.defaults.DefaultSqlSession@13521226] was not registered for synchronization because synchroni
2:57.321 INFO 15060 --- [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2:57.557 INFO 15060 --- [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed
[HikariProxyConnection@183243451 wrapping com.mysql.cj.jdbc.ConnectionImpl@245aded6] will not be managed by Spring
select * from dept

id, name, create_time, update_time
3, 咨询部, 2022-12-30 13:53:04, 2022-12-30 13:53:04
4, 就业部, 2022-12-30 13:53:04, 2022-12-30 13:53:04
2
nsactional SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@13521226]
2:57.604 INFO 15060 --- [nio-8080-exec-1] com.itheima.aspect.MyAspect6 : MyAspect6 -> after ...

```

到此我们两种常见的切入点表达式我已经介绍完了。

- execution切入点表达式
 - 根据我们所指定的方法的描述信息来匹配切入点方法，这种方式也是最为常用的一种方式
 - 如果我们要匹配的切入点方法的方法名不规则，或者有一些比较特殊的需求，通过execution切入点表达式描述比较繁琐
- annotation 切入点表达式
 - 基于注解的方式来匹配切入点方法。这种方式虽然多一步操作，我们需要自定义一个注解，但是相对来比较灵活。我们需要匹配哪个方法，就在方法上加上对应的注解就可以了



根据业务需要，可以使用 && ， || ， ! 来组合比较复杂的切入点表达式。

3. AOP案例

SpringAOP的相关知识我们就已经全部学习完毕了。最后我们要通过一个案例来对AOP进行一个综合的应用。

3.1 需求

需求：将案例（Tlias智能学习辅助系统）中增、删、改相关接口的操作日志记录到数据库表中

- 就是当访问部门管理和员工管理当中的增、删、改相关功能接口时，需要详细的操作日志，并保存在数据表中，便于后期数据追踪。

操作日志信息包含：

- 操作人、操作时间、执行方法的全类名、执行方法名、方法运行时参数、返回值、方法执行时长

所记录的日志信息包括当前接口的操作人是谁操作的，什么时间点操作的，以及访问的是哪个类当中的哪个方法，在访问这个方法的时候传入进来的参数是什么，访问这个方法最终拿到的返回值是什么，以及整个接口方法的运行时长是多长时间。

3.2 分析

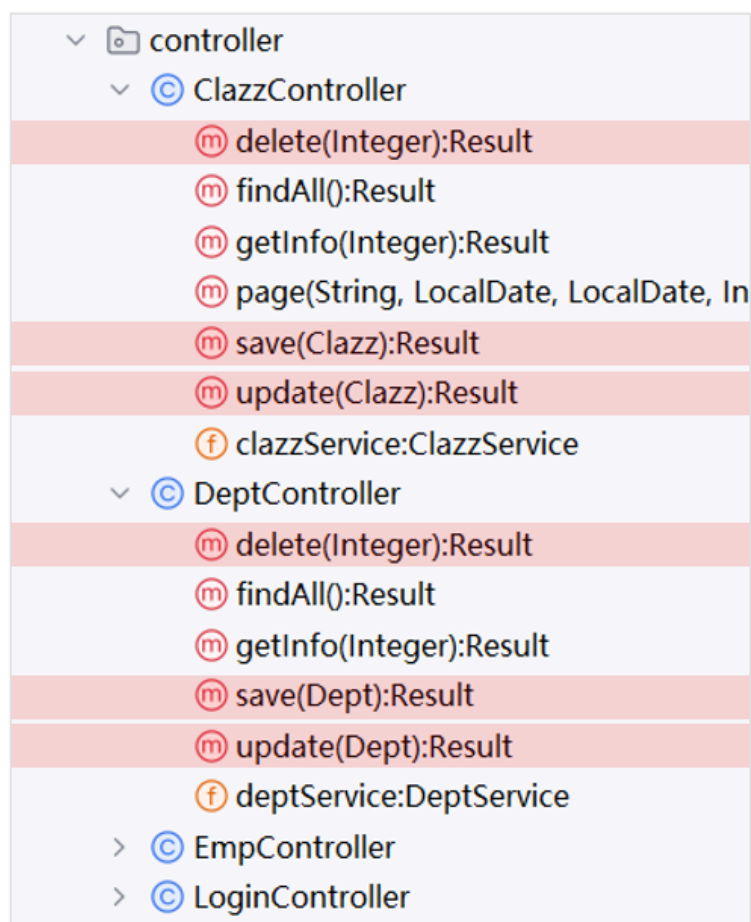
- 问题1：项目当中增删改相关的方法是不是有很多？
 - 很多
- 问题2：我们需要针对每一个功能接口方法进行修改，在每一个功能接口当中都来记录这些操作日志吗？
 - 这种做法比较繁琐

以上两个问题的解决方案：可以使用AOP解决(每一个增删改功能接口中要实现的记录操作日志的逻辑代码是相同)。

可以把这部分记录操作日志的通用的、重复性的逻辑代码抽取出来定义在一个通知方法当中，我们通过AOP面向切面编程的方式，在不改动原始功能的基础上来对原始的功能进行增强。目前我们所增强的功能就是来记录操作日志，所以也可以使用AOP的技术来实现。使用AOP的技术来实现也是最为简单，最为方便的。

- 问题3：既然要基于AOP面向切面编程的方式来完成的功能，那么我们要使用 AOP五种通知类型当中的哪种通知类型？

- 答案：环绕通知 `@Around`。因为所记录的操作日志当中包括：操作人、操作时间，访问的是哪个类、哪个方法、方法运行时参数、方法的返回值、方法的运行时长。方法返回值，是在原始方法执行后才能获取到的。方法的运行时长，需要原始方法运行之前记录开始时间，原始方法运行之后记录结束时间。通过计算获得方法的执行耗时。基于以上的分析我们确定要使用 `Around` 环绕通知。
- 问题4：最后一个问题，切入点表达式我们该怎么写？
 - 答案：使用 `@annotation` 来描述切入点表达式。要匹配业务接口当中所有的增删改的方法，而增删改方法在命名上没有共同的前缀或后缀。此时如果使用 `execution` 切入点表达式也可以，但是会比较繁琐。当遇到增删改的方法名没有规律时，就可以使用 `@annotation` 切入点表达式



3.3 步骤

简单分析了一下大概的实现思路后，接下来我们就要来完成案例了。案例的实现步骤其实就两步：

- 准备工作
 - a. 引入AOP的起步依赖
 - b. 导入资料中准备好的数据库表结构，并引入对应的实体类

- 编码实现(基于AOP实现)
 - a. 自定义注解 `@LogOperation`
 - b. 定义切面类，完成记录操作日志的逻辑

3.4 代码实现

 请帮我基于Spring AOP中的环绕通知 `@Around` 实现记录系统所有增、删、改功能接口的操作日志。具体信息如下：

1. 日志信息包含：操作人、操作时间、执行方法的全类名、执行方法名、方法运行时参数、返回值、方法执行时长
2. 功能接口所在包为 `com.itheima.controller`
3. 日志表为 `operate_log` 表，对应的实体类为 `OperateLog`。具体表结构如下：

```
create table operate_log(  
    id int unsigned primary key auto_increment comment 'ID',  
    operate_emp_id int unsigned comment '操作人ID',  
    operate_time datetime comment '操作时间',  
    class_name varchar(100) comment '操作的类名',  
    method_name varchar(100) comment '操作的方法名',  
    method_params varchar(1000) comment '方法参数',  
    return_value varchar(2000) comment '返回值',  
    cost_time int comment '方法执行耗时, 单位:ms'  
) comment '操作日志表';
```

4. 并且已经提供了 `OperateLogMapper` 接口来操作 `operate_log`, 并在其中已经定义好了 `insert` 方法用来保存日志数据。

1). 准备工作

- 在 `pom.xml` 中引入AOP的依赖

代码块

```
1 <dependency>  
2     <groupId>org.springframework.boot</groupId>  
3     <artifactId>spring-boot-starter-aop</artifactId>
```


- 创建数据库表结构

代码块

```

1  -- 操作日志表
2  create table operate_log(
3      id int unsigned primary key auto_increment comment 'ID',
4      operate_emp_id int unsigned comment '操作人ID',
5      operate_time datetime comment '操作时间',
6      class_name varchar(100) comment '操作的类名',
7      method_name varchar(100) comment '操作的方法名',
8      method_params varchar(1000) comment '方法参数',
9      return_value varchar(2000) comment '返回值, 存储json格式',
10     cost_time int comment '方法执行耗时, 单位:ms'
11 ) comment '操作日志表';

```

- 引入资料中准备的实体类

代码块

```

1  package com.itheima.pojo;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Data;
5  import lombok.NoArgsConstructor;
6  import java.time.LocalDateTime;
7
8  @Data
9  @NoArgsConstructor
10 @AllArgsConstructor
11 public class OperateLog {
12     private Integer id; //ID
13     private Integer operateEmpId; //操作人ID
14     private LocalDateTime operateTime; //操作时间
15     private String className; //操作类名
16     private String methodName; //操作方法名
17     private String methodParams; //操作方法参数
18     private String returnValue; //操作方法返回值
19     private Long costTime; //操作耗时
20 }
21

```

- 引入资料中准备的日志操作Mapper接口 `OperateLogMapper`

代码块

```
1 package com.itheima.mapper;
2
3 import com.itheima.pojo.OperateLog;
4 import org.apache.ibatis.annotations.Insert;
5 import org.apache.ibatis.annotations.Mapper;
6
7 @Mapper
8 public interface OperateLogMapper {
9
10     //插入日志数据
11     @Insert("insert into operate_log (operate_emp_id, operate_time,
12         class_name, method_name, method_params, return_value, cost_time) " +
13         "values ({operateEmpId}, #{operateTime}, #{className}, #{
14         methodName}, #{methodParams}, #{returnValue}, #{costTime});")
15     public void insert(OperateLog log);
16 }
```

1). 自定义注解 @LogOperation

代码块

```
1 /**
2  * 自定义注解，用于标识哪些方法需要记录日志
3  */
4 @Target(ElementType.METHOD)
5 @Retention(RetentionPolicy.RUNTIME)
6 public @interface LogOperation {
7 }
```

2). 定义AOP记录日志的切面类

代码块

```
1 import com.itheima.anno.LogOperation;
2 import com.itheima.mapper.OperateLogMapper;
3 import com.itheima.pojo.OperateLog;
4 import org.aspectj.lang.ProceedingJoinPoint;
```

```
5 import org.aspectj.lang.annotation.Around;
6 import org.aspectj.lang.annotation.Aspect;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Component;
9 import java.time.LocalDateTime;
10 import java.util.Arrays;
11
12 @Aspect
13 @Component
14 public class OperationLogAspect {
15
16     @Autowired
17     private OperateLogMapper operateLogMapper;
18
19     // 环绕通知
20     @Around("@annotation(log)")
21     public Object around(ProceedingJoinPoint joinPoint, LogOperation log)
22     throws Throwable {
23         // 记录开始时间
24         long startTime = System.currentTimeMillis();
25         // 执行方法
26         Object result = joinPoint.proceed();
27         // 当前时间
28         long endTime = System.currentTimeMillis();
29         // 耗时
30         long costTime = endTime - startTime;
31
32         // 构建日志对象
33         OperateLog operateLog = new OperateLog();
34         operateLog.setOperateEmpId(getCurrentUserId()); // 需要实现
35         // 获取当前用户ID 方法
36         operateLog.setOperateTime(LocalDateTime.now());
37         operateLog.setClassName(joinPoint.getTarget().getClass().getName());
38         operateLog.setMethodName(joinPoint.getSignature().getName());
39         operateLog.setMethodParams(Arrays.toString(joinPoint.getArgs()));
40         operateLog.setReturnValue(result.toString());
41         operateLog.setCostTime(costTime);
42
43         // 插入日志
44         operateLogMapper.insert(operateLog);
45         return result;
46     }
47
48     // 示例方法，获取当前用户ID
49     private int getCurrentUserId() {
50         // 这里应该根据实际情况从认证信息中获取当前登录用户的ID
51         return 1; // 示例返回值
52     }
53 }
```

```
50     }
51 }
```

3). 在需要记录的日志的Controller层的方法上，加上注解 `@LogOperation`

代码块

```
1  @RestController
2  @RequestMapping("/clazzs")
3  public class ClazzController {
4
5      @Autowired
6      private ClazzService clazzService;
7
8      /**
9       * 新增班级
10      */
11      @LogOperation
12      @PostMapping
13      public Result save(@RequestBody Clazz clazz){
14          clazzService.save(clazz);
15          return Result.success();
16      }
17 }
```

重启SpringBoot服务，测试操作日志记录功能：

打开浏览器，针对于员工的数据、部门的数据进行增删改之后。我们打开数据库表结构可以来看一下：

WHERE							ORDER BY	
id	operate_emp_id	operate_time	class_name	method...	method_params			
1	1	1 2024-08-19 14:36:06	com.itheima.controller.DeptController	save	[Dept(id=null, name=总经办, crea			
2	2	1 2024-08-19 14:36:11	com.itheima.controller.DeptController	save	[Dept(id=null, name=后勤部, crea			
3	3	1 2024-08-19 14:36:13	com.itheima.controller.DeptController	delete	[12]			
4	4	1 2024-08-19 14:36:18	com.itheima.controller.DeptController	update	[Dept(id=13, name=后勤部门, crea			
5	5	1 2024-08-19 14:36:28	com.itheima.controller.ClazzController	update	[Clazz(id=9, name=JavaEE就业168			
6	6	1 2024-08-19 14:38:27	com.itheima.controller.DeptController	delete	[13]			
7	7	1 2024-08-19 14:43:36	com.itheima.controller.DeptController	save	[Dept(id=null, name=部门表, crea			
8	8	1 2024-08-19 14:43:38	com.itheima.controller.DeptController	delete	[14]			

我们会看到，在数据库表中，就清晰的记录了谁、什么时间点、调用了哪个类的哪个方法、传入了什么参数、返回了什么数据，都清晰的记录在数据库中了。

3.5 连接点

我们前面在讲解AOP核心概念的时候，我们提到过什么是连接点，连接点可以简单理解为可以被AOP控制的方法。

我们目标对象当中所有的方法是不是都是可以被AOP控制的方法。而在SpringAOP当中，连接点又特指方法的执行。

在Spring中用JoinPoint抽象了连接点，用它可以获得方法执行时的相关信息，如目标类名、方法名、方法参数等。

- 对于 `@Around` 通知，获取连接点信息只能使用 `ProceedingJoinPoint` 类型

@Around环绕通知

```
@Around("execution(* com.itheima.service.DeptService.*(..))")
public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
    String className = joinPoint.getTarget().getClass().getName(); //获取目标类名
    Signature signature = joinPoint.getSignature(); //获取目标方法签名
    String methodName = joinPoint.getSignature().getName(); //获取目标方法名
    Object[] args = joinPoint.getArgs(); //获取目标方法运行参数
    Object res = joinPoint.proceed(); //执行原始方法,获取返回值（环绕通知）
    return res;
}
```

- 对于其他四种通知，获取连接点信息只能使用 `JoinPoint`，它是 `ProceedingJoinPoint` 的父类型

```

@Before("execution(* com.itheima.service.DeptService.*(..))")
public void before(JoinPoint joinPoint) {
    String className = joinPoint.getTarget().getClass().getName(); //获取目标类名
    Signature signature = joinPoint.getSignature(); //获取目标方法签名
    String methodName = joinPoint.getSignature().getName(); //获取目标方法名
    Object[] args = joinPoint.getArgs(); //获取目标方法运行参数
}

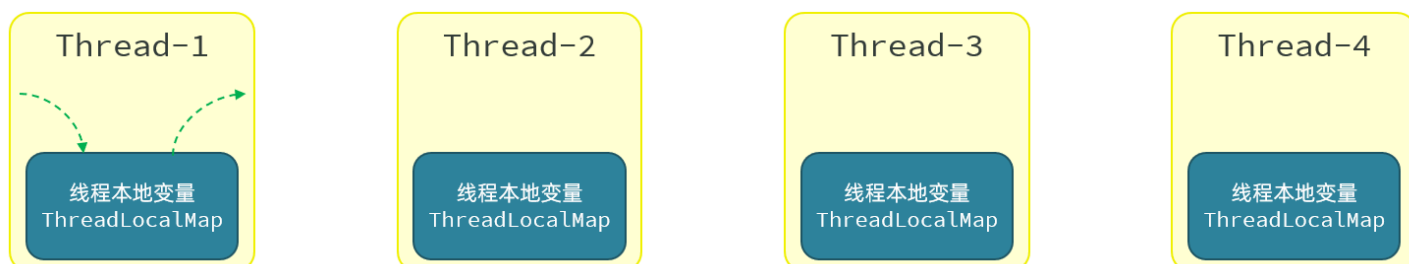
```

3.6 获取当前登录员工

- 员工登录成功后，哪里存储的有当前登录员工的信息？ 给客户端浏览器下发的jwt令牌中
- 如何从JWT令牌中获取当前登录用户的信息呢？ 获取请求头中传递的jwt令牌，并解析
- TokenFilter 中已经解析了令牌的信息，如何传递给AOP程序、Controller、Service呢？
ThreadLocal

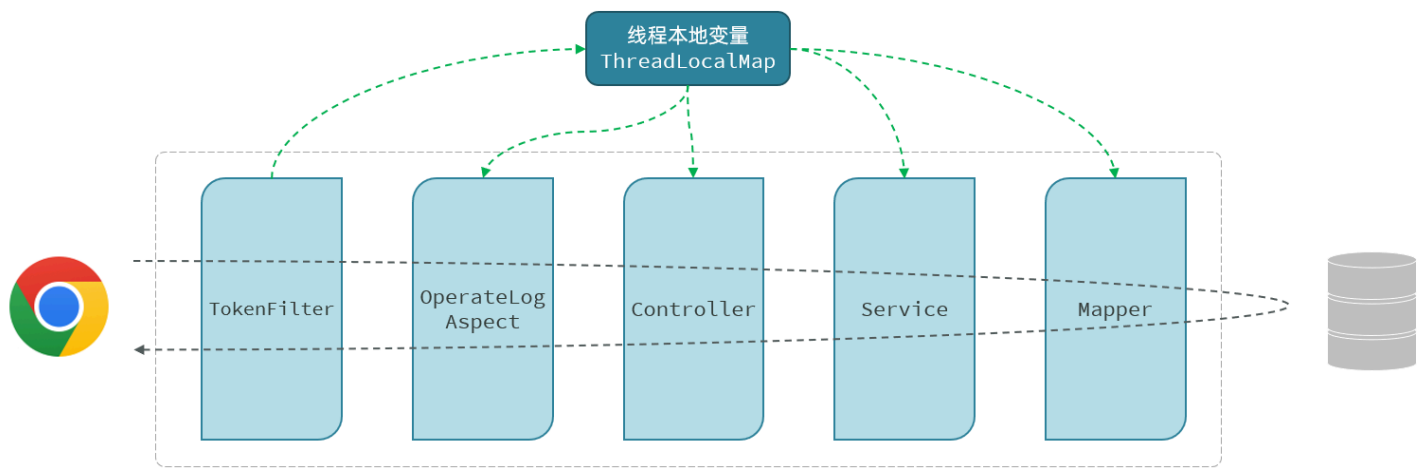
3.6.1 ThreadLocal

- ThreadLocal 并不是一个Thread，而是Thread的局部变量。
- ThreadLocal为每个线程提供一份单独的存储空间，具有线程隔离的效果，不同的线程之间不会相互干扰。



- 常见方法：
 - `public void set(T value)` 设置当前线程的线程局部变量的值
 - `public T get()` 返回当前线程所对应的线程局部变量的值
 - `public void remove()` 移除当前线程的线程局部变量

3.6.2 记录当前登录员工



具体操作步骤：

1. 定义ThreadLocal操作的工具类，用于操作当前登录员工ID。

在 `com.itheima.utils` 引入工具类 `CurrentHolder`

代码块

```
1 package com.itheima.utils;
2
3 public class CurrentHolder {
4
5     private static final ThreadLocal<Integer> CURRENT_LOCAL = new ThreadLocal<>
6     ();
7
8     public static void setCurrentId(Integer employeeId) {
9         CURRENT_LOCAL.set(employeeId);
10    }
11
12    public static Integer getCurrentId() {
13        return CURRENT_LOCAL.get();
14    }
15
16    public static void remove() {
17        CURRENT_LOCAL.remove();
18    }
19 }
```

2. 在 `TokenFilter` 中，解析完当前登录员工ID，将其存入ThreadLocal（用完之后需将其删除）。

代码块

```
1 package com.itheima.filter;
2
3 import com.itheima.utils.CurrentHolder;
```

```
4 import com.itheima.utils.JwtUtils;
5 import io.jsonwebtoken.Claims;
6 import jakarta.servlet.*;
7 import jakarta.servlet.annotation.WebFilter;
8 import jakarta.servlet.http.HttpServletRequest;
9 import jakarta.servlet.http.HttpServletResponse;
10 import lombok.extern.slf4j.Slf4j;
11 import java.io.IOException;
12
13 @Slf4j
14 @WebFilter(urlPatterns = "/*")
15 public class TokenFilter implements Filter {
16     @Override
17     public void doFilter(ServletRequest servletRequest, ServletResponse
servletResponse, FilterChain filterChain) throws IOException, ServletException
{
18         HttpServletRequest request = (HttpServletRequest) servletRequest;
19         HttpServletResponse response = (HttpServletResponse) servletResponse;
20
21         //1. 获取请求的url地址
22         String uri = request.getRequestURI(); // /employee/login
23         //String url = request.getRequestURL().toString(); //
http://localhost:8080/employee/login
24
25         //2. 判断是否是登录请求, 如果url地址中包含 login, 则说明是登录请求, 放行
26         if (uri.contains("login")) {
27             log.info("登录请求, 放行");
28             filterChain.doFilter(request, response);
29             return;
30         }
31
32         //3. 获取请求中的token
33         String token = request.getHeader("token");
34
35         //4. 判断token是否为空, 如果为空, 响应401状态码
36         if (token == null || token.isEmpty()) {
37             log.info("token为空, 响应401状态码");
38             response.setStatus(401); // 响应401状态码
39             return;
40         }
41
42         //5. 如果token不为空, 调用JwtUtils工具类的方法解析token, 如果解析失败, 响应
401状态码
43         try {
44             Claims claims = JwtUtils.parseJWT(token);
45             Integer empId = Integer.valueOf(claims.get("id").toString());
46             CurrentHolder.setCurrentId(empId);
```



```

47         log.info("token解析成功，放行");
48     } catch (Exception e) {
49         log.info("token解析失败，响应401状态码");
50         response.setStatus(401);
51         return;
52     }
53
54     //6. 放行
55     filterChain.doFilter(request, response);
56
57     //7. 清空当前线程绑定的id
58     CurrentHolder.remove();
59 }
60 }

```

3. 在AOP程序中，从ThreadLocal中获取当前登录员工的ID。

代码块

```

1  package com.itheima.aop;
2
3  import com.itheima.anno.LogOperation;
4  import com.itheima.mapper.OperateLogMapper;
5  import com.itheima.pojo.OperateLog;
6  import com.itheima.utils.CurrentHolder;
7  import org.aspectj.lang.ProceedingJoinPoint;
8  import org.aspectj.lang.annotation.Around;
9  import org.aspectj.lang.annotation.Aspect;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.stereotype.Component;
12
13 import java.time.LocalDateTime;
14 import java.util.Arrays;
15
16 @Aspect
17 @Component
18 public class OperationLogAspect {
19
20     @Autowired
21     private OperateLogMapper operateLogMapper;
22
23     // 环绕通知
24     @Around("@annotation(log)")
25     public Object around(ProceedingJoinPoint joinPoint, LogOperation log)
26     throws Throwable {

```

```

26      // 记录开始时间
27      long startTime = System.currentTimeMillis();
28      // 执行方法
29      Object result = joinPoint.proceed();
30      // 当前时间
31      long endTime = System.currentTimeMillis();
32      // 耗时
33      long costTime = endTime - startTime;
34
35      // 构建日志对象
36      OperateLog operateLog = new OperateLog();
37      operateLog.setOperateEmpId(getCurrentUserId()); // 需要实现
    getCurrentUserId 方法
38      operateLog.setOperateTime(LocalDateTime.now());
39      operateLog.setClassName(joinPoint.getTarget().getClass().getName());
40      operateLog.setMethodName(joinPoint.getSignature().getName());
41      operateLog.setMethodParams(Arrays.toString(joinPoint.getArgs()));
42      operateLog.setReturnValue(result.toString());
43      operateLog.setCostTime(costTime);
44
45      // 插入日志
46      operateLogMapper.insert(operateLog);
47      return result;
48  }
49
50      // 示例方法，获取当前用户ID
51      private int getCurrentUserId() {
52          return CurrentHolder.getCurrentId();
53      }
54  }

```

代码优化完毕之后，我们重新启动服务测试。就可以看到，可以获取到不同的登录用户信息了。

tlas operate_log x						
12 rows						
Tx: Auto DDL						
WHERE ORDER BY						
	id	operate_emp_id	operate_time	class_name	method_...	method_params
1	1	1	2024-08-19 14:36:06	com.itheima.controller.DeptController	save	[Dept(id=null, name=总经办, cr
2	2	1	2024-08-19 14:36:11	com.itheima.controller.DeptController	save	[Dept(id=null, name=后勤部, cr
3	3	1	2024-08-19 14:36:13	com.itheima.controller.DeptController	delete	[12]
4	4	1	2024-08-19 14:36:18	com.itheima.controller.DeptController	update	[Dept(id=13, name=后勤部门, cr
5	5	1	2024-08-19 14:36:28	com.itheima.controller.ClazzController	update	[Clazz(id=9, name=JavaEE就业1
6	6	1	2024-08-19 14:38:27	com.itheima.controller.DeptController	delete	[13]
7	7	1	2024-08-19 14:43:36	com.itheima.controller.DeptController	save	[Dept(id=null, name=部门表, cr
8	8	1	2024-08-19 14:43:38	com.itheima.controller.DeptController	delete	[14]
9	9	1	2024-08-19 15:08:46	com.itheima.controller.DeptController	update	[Dept(id=1, name=学工部7, crea
10	10	1	2024-08-19 15:08:49	com.itheima.controller.DeptController	update	[Dept(id=1, name=学工部, creat
11	11	10	2024-08-19 15:18:58	com.itheima.controller.DeptController	update	[Dept(id=8, name=人事部2, crea
12	12	10	2024-08-19 15:19:00	com.itheima.controller.DeptController	update	[Dept(id=8, name=人事部, creat



在同一个线程/同一个请求中，进行数据共享就可以使用 ThreadLocal。